第一部分：函数基础与函数参数

Slide 3: 什么是函数?

- 定义：函数是具备某一功能的工具，类似于工厂中的工具（如锤子、扳手）。
- 原则：先定义，后调用。
- 类比：工厂制造工具，工具才能使用。

Slide 4: 为何要用函数?

- 问题：
  1. 程序冗长
  2. 扩展性差
  3. 可读性差
- 解决方案：使用函数提高代码的模块化和可维护性。

Slide 5: 如何定义函数

- 语法：

```python
def 函数名():
    函数体代码
```

- 示例：

```python
def factory():
    print('正在制造手机')
    print('正在制造电脑')
```

Slide 6: 函数调用

- 调用方式：函数名加括号 `factory()`
- 示例：

```python
factory()
factory()
```

Slide 7: 函数的三大特性

1. 功能：函数体代码
2. 参数：传递外部数据
3. 返回值：返回处理结果

Slide 8: 无参函数

- 应用场景：执行简单操作，如打印。
- 示例：

```python
def factory():
    print('正在制造手机')
```

Slide 9: 有参函数

- 形参：定义函数时的变量名。
- 实参：调用函数时传入的值。
- 示例：

```python
def factory(a, b):
    print(f'a是{a}, b是{b}')
factory(1, 2)
```

Slide 10: 位置参数与关键字参数

- 位置参数：按顺序传值。
- 关键字参数：按 `key=value` 传值。
- 示例：

```python
factory(a=1, b=2)
factory(b=2, a=1)
```

Slide 11: 默认参数

- 定义：在定义函数时为形参赋值。
- 示例：

```python
def factory(a, b=2):
    print(a + b)
factory(1)
```

Slide 12: 可变长度参数

- `*args`：接收多余的位置参数，保存为元组。
- `**kwargs`：接收多余的关键字参数，保存为字典。
- 示例：

```python
def factory(*args, **kwargs):
    print(args)
    print(kwargs)
factory(1, 2, x=3, y=4)
```

Slide 13: 命名关键字参数

- 定义：位于 `*args` 和 `**kwargs` 之间的参数。
- 示例：

```python
def foo(x, y, *args, m, n, **kwargs):
    print(x, y, args, m, n, kwargs)
foo(1, 2, 3, 4, m=100, n=200, a=1, b=2)
```